

Checkpoint 4

1. ¿Cuál es la diferencia entre una lista y una tupla en Python?

Ambas son colecciones de elementos que pueden ser de igual o diferente tipo.

La diferencia es que una lista se puede modificar y una tupla no.

Una lista se declara entre corchetes [], mientras que para una tupla se utilizan paréntesis ()

Si una tupla tiene un elemento mutable, como una lista, este elemento sí se puede modificar, lo que no se puede es cambiar el elemento en sí.

Ejemplo:

- Lista:

```
lista = ['Uno', 2, 'Tres', ['A', 'B', 'C', 'D'], 'Cinco']
```

```
lista[1] = 'Dos' → esto es correcto
```

- Tupla:

```
tupla = ('Uno', 2, 'Tres', ['A', 'B', 'C', 'D'], 'Cinco')
```

```
tupla[1] = 'Dos' → esto NO es correcto
```

```
tupla[3] = [20] → esto NO es correcto
```

```
tupla[3][1] = 'J' → esto es correcto
```

2. ¿Cuál es el orden de las operaciones?

Las operaciones se ordenan de la siguiente forma:

- Paréntesis
 - Exponente
 - Multiplicación
 - División
 - Suma
 - Resta
- } El orden es intercambiable
- } El orden es intercambiable

Checkpoint 4

3. ¿Qué es un diccionario Python?

Es una estructura de datos similar a una lista con el formato clave-valor.

Así como en una lista se utiliza el índice para acceder a un elemento, en un diccionario se utiliza una clave, que es única (no puede repetirse) para acceder al valor del elemento.

Al igual que la lista, también se puede modificar.

Se declara entre llaves {} y los elementos se separan por comas ','.

Ejemplo:

```
lista=[1,'dos',3.5,['A', 'B', 'C', 'D']]
```

```
diccionario= {  
    'primero': 1,  
    'segundo': 'dos',  
    'tercero': 3.5,  
    'cuarto' : ['A', 'B', 'C', 'D'],  
}
```

```
print(lista[3]) → ['A', 'B', 'C', 'D']
```

```
print(diccionario['cuarto']) → ['A', 'B', 'C', 'D']
```

```
diccionario['cuarto'] = 'String'
```

```
print(diccionario) → {'primero': 1, 'segundo': 'dos', 'tercero': 3.5, 'cuarto':  
'String'}
```

Checkpoint 4

4. ¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

El método ordenado modifica la variable original y la función no.

```
lista=[1, 2, 3, 4]  
lista_ordenada=sorted(lista, reverse=True)
```

```
print(lista) → [1, 2, 3, 4]
```

```
print(lista_ordenada) → [4, 3, 2, 1]
```

```
lista.sort(reverse=True)
```

```
print(lista) → [4, 3, 2, 1]
```

5. ¿Qué es un operador de reasignación?

Es una forma abreviada de actualizar el valor de una variable.

En lugar de escribir toda la operación y asignarla a la misma variable se usa un operador seguido de '=' y un valor.

```
Variable = 5
```

```
variable *= 2 → 10 (reasignación)
```

```
variable = variable*2 → 20
```